与えられたグラフに対して、任意の 2 頂点間の最短経路を計算するアルゴリズムを書いた。

入力：グラフを表す隣接頂点関数 T、頂点 v, w 間の枝の重みを返す関数 $l_{v,w}$、始点 s、終点 t

出力：最短経路を表す頂点列

最短経路探索のアルゴリズムを以下に示す。

s から頂点 v までの最短経路長を表す関数 d_v

s から頂点 v までの最短経路を考えたとき、v の直前の頂点を表す関数 $prev_v$

展開する頂点を格納する集合 A

を用意する。

1. $d_s = 0$, s 以外のすべての頂点に対し $d_v = \infty$ とする。

2. s を A に追加。

3. A に含まれる頂点のうち d_v の値が最小となる頂点 v を取り出す。（A からは消す）

4. v に隣接する各頂点 v に対して、

if $d_w = \infty$ then $d_w = d_v + l_{v,w}$ とし w を A に追加。$prev_w = v$ elseif $d_w > d_v + l_{v,w}$ then $d_w = d_v + l_{v,w}$, $prev_w = v$

5. A が空なら手順 6 へ、空でないなら手順 3 へ戻る。6. $prev_t$ から辿って行けば t から s への経路が得られるのでこれを出力する。最短経路長は d_t で与えられる。

これで最短距離が求まる正当性を簡単に説明する。

まず各頂点は「距離が確定した頂点」と「距離が確定していない頂点」に分類できる。「距離が確定した頂点」は手順 3 で A から取り出されたことのある頂点集合である。手順 3 を行う直前の時点ですべての d_v は、「距離が確定した頂点」から直接その頂点に移動したときの最短距離以下になっている。（手順 4 により）実はここで「距離が確定していない頂点」のうち、現時点でもっとも d_v の値が小さい頂点は、その d_v の値を確定させて良い。手順 4 のような更新を喰らうことがもうないため、これ以上 d_v を小さくすることができないからである。このようにして最短距離がドミノ式に確定していく。

A はヒープによって実装した。ヒープの各頂点は「頂点番号」と「最短距離」という 2 次元の情報を持ち、「最短距離」によって比較される。A の内部にある最短距離情報を直接更新する必要はなく、新たな「頂点番号」「最短距離」の 2 次元の値をヒープに追加すれば上記のアルゴリズムが正当に動く。